

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Repaso pre-parcial

Ejercicios para resolver en el pizarrón

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Introducción

Sobre la clase de hoy

Cuatro ejercicios que recorren los temas del parcial, para resolver juntos en el pizarrón

Sobre la clase de hoy

Cuatro ejercicios que recorren los temas del parcial, para resolver juntos en el pizarrón

1. **Representación y aritmética**

Sobre la clase de hoy

Cuatro ejercicios que recorren los temas del parcial, para resolver juntos en el pizarrón

1. **Representación y aritmética**
2. **Combinacionales y diseño jerárquico**

Sobre la clase de hoy

Cuatro ejercicios que recorren los temas del parcial, para resolver juntos en el pizarrón

1. **Representación y aritmética**
2. **Combinacionales y diseño jerárquico**
3. **Secuenciales y FSM**

Sobre la clase de hoy

Cuatro ejercicios que recorren los temas del parcial, para resolver juntos en el pizarrón

1. **Representación y aritmética**
2. **Combinacionales y diseño jerárquico**
3. **Secuenciales y FSM**
4. **Camino de datos, control e ISA**

Convenciones

- Se conservan los bits del ancho indicado
- **CA2** significa complemento a dos
- Flags del sumador:
  - **C**: carry
  - **V**: overflow en CA2
  - **Z**: el resultado es cero
  - **N**: bit más significativo del resultado
- Los registros capturan simultáneamente en el flanco ascendente
- El reset es sincrónico y activo en 1

Ejercicio 1: representación y aritmética

Ejercicio 1: enunciado

Un dispositivo recibe palabras de **6 bits**. Se consideran:

$$A = \texttt{011101} \qquad B = \texttt{000111} \qquad D = \texttt{111101}$$

Ejercicio 1: enunciado

Un dispositivo recibe palabras de **6 bits**. Se consideran:

$$A = \boxed{011101} \qquad B = \boxed{000111} \qquad D = \boxed{111101}$$

1. Interpretar cada patrón **sin signo** y en **CA2**. Escribir también su patrón en **hexadecimal**, aclarando que el ancho original sigue siendo de 6 bits

Ejercicio 1: sumas de 6 bits

$A = \boxed{\texttt{011101}}$ $B = \boxed{\texttt{000111}}$ $D = \boxed{\texttt{111101}}$

2. Calcular $A + B$ y $D + B$ con un sumador de 6 bits
   - Mostrar la suma binaria, el resultado almacenado y los flags **CVZN**
   - Determinar, en ambas interpretaciones, si el resultado almacenado representa la suma matemática

Ejercicio 1: sumas de 8 bits

$A = \boxed{011101}$ $B = \boxed{000111}$ $D = \boxed{111101}$

3. Repetir ambas sumas con **8 bits**, extendiendo primero los operandos de manera apropiada para cada interpretación
   - Mostrar operandos extendidos, resultado y flags
   - Explicar qué problemas de representabilidad desaparecen
   - Explicar por qué un carry no implica necesariamente un resultado incorrecto en CA2

Ejercicio 2: combinacionales y diseño jerárquico

Ejercicio 2: enunciado

Diseñar un circuito combinacional con entradas a y b de **6 bits en CA2**, control `modo` y salida y de 6 bits:

- `modo` = 0: entrega el **menor**
- `modo` = 1: entrega el **mayor**
- Si son iguales, puede seleccionar cualquiera

Ejercicio 2: enunciado

Diseñar un circuito combinacional con entradas a y b de **6 bits en CA2**, control `modo` y salida y de 6 bits:

- `modo` = 0: entrega el **menor**
- `modo` = 1: entrega el **mayor**
- Si son iguales, puede seleccionar cualquiera

Se dispone de un **restador** que calcula $a - b$ y entrega los flags **N** y **V**

- **N** es el bit de signo del resultado
- **V** indica overflow en CA2

Ejercicio 2: señales de control

1. Completar la tabla de `menor` en función de N y V y obtener una expresión
   - ¿Qué sucede con el signo del resultado cuando la resta desborda?
   - Derivar después `elige_a` en función de `modo` y `menor`

N	V	`menor`: $a < b$
0	0	
0	1	
1	0	
1	1	

Ejercicio 2: el módulo

2. Completar el módulo **sin usar** < ni >. Los flags N y V provienen del restador externo aplicado a esas mismas entradas a y b

```
module minmax6 (
    input  logic [5:0] a, b,
    input  logic       modo, N, V,
    output logic [5:0] y
);
    logic menor, elige_a;
    assign menor   = ______;
    assign elige_a = ______;
    assign y       = ______;
endmodule
```

Ejercicio 2: el circuito

3. Dibujar el circuito completo, incluyendo:
   - el restador
   - la lógica de selección
   - los multiplexores

   Indicar el **ancho** de las conexiones

Ejercicio 3: secuenciales y FSM

Ejercicio 3: enunciado

Un sensor entrega un bit x por ciclo. El sistema activa una **alarma** al observar **dos muestras consecutivas** iguales a 1. Una vez activada, permanece encendida hasta recibir `ack` = 1

Ejercicio 3: enunciado

Un sensor entrega un bit x por ciclo. El sistema activa una **alarma** al observar **dos muestras consecutivas** iguales a 1. Una vez activada, permanece encendida hasta recibir `ack` = 1

- El **reset** apaga la alarma y borra toda coincidencia parcial
- En cualquier estado, `ack` **tiene prioridad**: en el flanco borra el avance y apaga la alarma, ignorando x
- La salida `alarma` debe ser **Moore**
- Un 0 interrumpe una coincidencia parcial, pero no apaga una alarma ya activada

Ejercicio 3: estados y transiciones

1. Definir los estados y explicar qué información recuerda cada uno

Ejercicio 3: estados y transiciones

1. Definir los estados y explicar qué información recuerda cada uno
2. Dibujar el diagrama completo y construir las tablas de transición y salidas, incluyendo el caso x = ack = 1

Ejercicio 3: salida Mealy

3. Agregar una salida **Mealy** `aviso`:
   - se activa en el mismo ciclo en que llega el segundo 1
   - permanece activa mientras la alarma esté enclavada
   - `ack` suprime `aviso` inmediatamente

Ejercicio 3: salida Mealy

3. Agregar una salida **Mealy** `aviso`:
   - se activa en el mismo ciclo en que llega el segundo 1
   - permanece activa mientras la alarma esté enclavada
   - `ack` suprime `aviso` inmediatamente

Completar la traza de la próxima slide con estado actual, próximo estado, `alarma` y `aviso`. Comparar **cuándo** cambian las dos salidas

Ejercicio 3: traza

El ciclo 0 comienza en el estado posterior al reset. Cada entrada se muestrea en el flanco que cierra su columna; el próximo estado aparece en el ciclo siguiente

Ciclo	0	1	2	3	4	5	6	7	8
x	1	0	1	1	0	1	1	1	0
`ack`	0	0	0	0	0	1	0	0	0
Estado actual									
Próximo estado									
`alarma` (Moore)									
`aviso` (Mealy)									

Ejercicio 4: camino de datos, control e ISA

Ejercicio 4: la máquina

Una máquina tiene:

- un acumulador **ACC** de 8 bits
- un registro temporal **T** de 8 bits
- una **ALU** combinacional que puede pasar su segundo operando, sumar, restar o hacer AND

Los resultados aritméticos conservan sus 8 bits bajos

Ejercicio 4: la ISA

Las instrucciones tienen 6 bits: `OO IIII`

`IIII` es un inmediato de 4 bits en CA2, que se **extiende con signo** a 8 bits, incluso para AND

OO	Instrucción	Efecto
`00`	`LOAD` inmediato	`ACC ← imm`
`01`	`ADD` inmediato	`ACC ← ACC + imm`
`10`	`SUB` inmediato	`ACC ← ACC - imm`
`11`	`AND` inmediato	`ACC ← ACC AND imm`

`imm`: inmediato extendido con signo a 8 bits

Ejercicio 4: ejecución

La instrucción ya está en **IR** y permanece estable durante su ejecución. Cada instrucción usa **dos estados**, uno por ciclo:

- **CALCULAR**: al terminar el ciclo, T captura el resultado de la ALU
- **GUARDAR**: al terminar el ciclo, ACC captura T

Ejercicio 4: ejecución

La instrucción ya está en **IR** y permanece estable durante su ejecución. Cada instrucción usa **dos estados**, uno por ciclo:

- **CALCULAR**: al terminar el ciclo, T captura el resultado de la ALU
- **GUARDAR**: al terminar el ciclo, ACC captura T
- Al reset, ACC = T = 0 y el controlador queda en CALCULAR
- Un mecanismo externo, que no se debe diseñar, presenta la primera instrucción antes de CALCULAR y actualiza IR después de cada flanco de GUARDAR, a tiempo para el siguiente ciclo
- No contar ciclos de búsqueda ni carga de IR

Ejercicio 4: camino de datos y control

1. Dibujar el **camino de datos**, incluyendo extensión de signo, ALU y conexiones entre registros. Identificar señales de **datos** y de **control**

Ejercicio 4: camino de datos y control

1. Dibujar el **camino de datos**, incluyendo extensión de signo, ALU y conexiones entre registros. Identificar señales de **datos** y de **control**
2. Completar la tabla de control con operación de ALU, `ldT` y `ldACC`. Indicar cómo se obtienen los campos desde IR y cómo transita el controlador

Estado	Operación ALU	`ldT`	`ldACC`	Efecto al flanco
CALCULAR				
GUARDAR				

Ejercicio 4: un programa

3. Codificar y ejecutar el programa. Mostrar ACC y T después de cada flanco e indicar el número total de ciclos

```
LOAD -3
ADD   7
SUB  -2
```

Ejercicio 4: un programa

3. Codificar y ejecutar el programa. Mostrar ACC y T después de cada flanco e indicar el número total de ciclos

```
LOAD -3
ADD   7
SUB  -2
```

4. Explicar qué sucede si, con el camino original, se activan `ldT` y `ldACC` en el primer flanco, intentando ejecutar una instrucción completa de una vez. Mostrar el caso de `LOAD -3` desde el reset

¿Preguntas?